

廈門大學



《汇编语言》实验报告

(三)

姓 名 王子恒

学 号 34520242201240

学 院 信息学院

专 业 软件工程

2025 年 10 月

1.实验目的

1. 熟练使用Debug（更加推荐采用程序形式），理解数据在内存中的存放，并理解并练习各种寻址方式。
2. 学习汇编语言的基本指令，学会阅读汇编代码并动手尝试编写一些算法
3. 学习虚拟机的使用方法，了解BIOS的基础运行方式，尝试破解BIOS密码

2.实验环境

Win11下，DosBox模拟汇编环境

3.实验内容

- 1) 在数据段中依次存入10H,11H,12H,13H,14H,15H,16H,17H，将其相加，并将结果存入DX寄存器。
- 2) 练习使用debug命令破解bios密码，写出自己对破解密码的理解。
- 3) （选做，部分指令知识需要预习）在长度为8的字节数组（无符号数）中，查找大于42H的无符号数的个数，存放在字节单元up中；等于42H的无符号数的个数，存放在字节单元equa中；小于42H的无符号数的个数，存放在字节单元down中。

程序显示up equa down的值。

八个数：31H,21H,42H,52H,87H,23H,98H,01H

4.实验具体实现

(1)在数据段中依次存入10H,11H,12H,13H,14H,15H,16H,17H，将其相加，并将结果存入DX寄存器。

代码

ASM lab3.asm

```
1  data segment
2      DB 10H,11H,12H,13H,14H,15H,16H,17H
3  data ends
4
5  code segment
6      assume cs:code, ds:data
7  start:
8      mov ax, data
9      mov ds, ax
10
11      mov ax, 0
12      mov cx, 8
13      mov si, 0
14
15  loop_add:
16      add al, [si]
17      inc si
18      loop loop_add
19
20      mov dx, ax
21
22      mov ah, 4CH
23      int 21H
24
25  code ends
26
27  end start
```

编译运行

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: lab3.asm
lab3.asm(25): error A2088: on form requires 80386/486

D:\>ml lab3.asm
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: lab3.asm

Microsoft (R) Segmented Executable Linker Version 5.60.339 Dec 5 1994
Copyright (C) Microsoft Corp 1984-1993. All rights reserved.

LINK : warning L4017: /r : unrecognized option name; option ignored
Object Modules [.obj]: /r lab3.obj
Run File [lab3.exe]: "lab3.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:
LINK : warning L4021: no stack segment

D:\>lab3.exe

D:\>debug lab3.exe
```

debug查看寄存器

先查看反汇编结果

```
D:\>debug lab3.exe
-u
076B:0000 B86A07      MOV     AX,076A
076B:0003 8ED8             MOV     DS,AX
076B:0005 B80000          MOV     AX,0000
076B:0008 B90800          MOV     CX,0008
076B:000B BE0000          MOV     SI,0000
076B:000E 0204           ADD     AL,[SI]
076B:0010 46             INC     SI
076B:0011 E2FB          LOOP    000E
076B:0013 8BD0          MOV     DX,AX
076B:0015 B44C          MOV     AH,4C
076B:0017 CD21          INT     21
076B:0019 D2E2          SHL     DL,CL
076B:001B FA          CLI
076B:001C 8946CE          MOV     [BP-32],AX
076B:001F 8956D0          MOV     [BP-30],DX
-
```

运行到0015的时候停止就能看到dx的值

```
D:\>debug.exe lab3.exe
-u
076B:0000 B86A07      MOV     AX,076A
076B:0003 8ED8             MOV     DS,AX
076B:0005 B80000          MOV     AX,0000
076B:0008 B90800          MOV     CX,0008
076B:000B BE0000          MOV     SI,0000
076B:000E 0204           ADD     AL,[SI]
076B:0010 46             INC     SI
076B:0011 E2FB          LOOP   000E
076B:0013 8BD0          MOV     DX,AX
076B:0015 B44C          MOV     AH,4C
076B:0017 CD21          INT     21
076B:0019 D2E2          SHL     DL,CL
076B:001B FA          CLI
076B:001C 8946CE        MOV     [BP-32],AX
076B:001F 8956D0        MOV     [BP-30],DX
-g 076B:0015

AX=009C  BX=0000  CX=0000  DX=009C  SP=0000  BP=0000  SI=0008  DI=0000
DS=076A  ES=075A  SS=0769  CS=076B  IP=0015  NV UP EI PL NZ NA PO NC
076B:0015 B44C          MOV     AH,4C
-
```

可以看到dx为0x9C，结果正确

(2) 练习使用debug命令破解bios密码，写出自己对破解密码的理解。

```
D:\>debug
-o 70 10
-i 71
3C
-o 71 ff
-i 71
7F
-q
D:\>
```

对破解密码的理解

破解密码的过程从技术角度来看，主要依赖于对计算机硬件底层操作的理解，特别是对 CMOS 芯片、BIOS 设置以及 I/O 端口的交互。通过这些操作，我们可以绕过 BIOS 密码验证机制。以下是我对破解密码的技术理解：

1. 密码存储与硬件交互原理

BIOS 密码通常存储在主板上的 CMOS 芯片中。CMOS 芯片通过两个 I/O 端口与 CPU 进行交互，分别是：

- **地址端口 (0x70)**：用于指定 CMOS 内部的存储地址。
- **数据端口 (0x71)**：用于读取或写入指定地址的数据。

在早期的 BIOS 系统中，密码信息通常以明文或简单加密形式存储在 CMOS 的特定地址（如 0x30、0x31、0x10 等）。通过直接操作这些地址，可以获取或修改存储在其中的密码信息。

2. 破解操作步骤与命令

通过调试工具（如 DOSBox 中的 `debug`），我们可以执行底层命令来操作 CMOS 芯片。以下是典型的命令及其含义：

- **读取密码：**
 1. `o 70 10`：向地址端口 0x70 写入 0x10，指定要访问的 CMOS 地址。
 2. `i 71`：从数据端口 0x71 读取该地址的内容。该地址存储了密码的部分信息。
- **修改密码：**
 1. `o 70 10`：再次向地址端口 0x70 写入 0x10，指定要访问的 CMOS 地址。
 2. `o 71 ff`：向数据端口 0x71 写入 0xFF，修改该地址的内容。这种修改会触发 BIOS 系统将密码重置为无效。

在具体操作中，`o` 命令用于输出数据到指定端口，`i` 命令用于从端口读取数据。通过这些基本命令的组合，攻击者可以读取密码或清除密码。

3. 技术细节：为什么不同地址有不同的作用

不同 BIOS 版本和不同厂商（如 Award、AMI）可能会将密码存储在不同的 CMOS 地址。例如，某些 BIOS 可能将密码分布在 0x30 和 0x31 地址，而其他 BIOS 可能将密码的标志存储在 0x10 地址。这种差异源于各厂商对 CMOS 地址的设计不同，因此具体地址的操作需要根据目标系统进行调整。

4. 现代 BIOS 安全机制的进步

现代计算机的 BIOS 密码保护已经发生了显著变化。相较于早期的简单存储方式，现代 BIOS 采用了更为复杂的加密方法来存储密码，且对 CMOS 地址的访问进行了限制。这些改进使得传统的密码破解方法（如通过直接操作 I/O 端口）已经失效。

- **加密存储**：密码通常经过加密存储，直接修改 CMOS 中的值已经无法破解密码。
- **校验和机制**：现代 BIOS 在存储数据时，通常会附加校验和（checksum），修改任意地址会导致校验失败，从而防止通过直接写入命令篡改密码数据。
- **端口保护**：部分主板对 CMOS 端口（0x70 和 0x71）进行限制，防止操作系统直接访问这些端口。

5. 总结

破解 BIOS 密码的过程本质上是通过操作计算机底层硬件来绕过安全验证。通过直接操作 CMOS 的地址端口，可以读取或修改存储在其中的密码数据。早期 BIOS 的密码存储机制较为简单，攻击者可以通过 debug 工具直接修改 CMOS 中的值，从而清除或绕过密码保护。然而，现代 BIOS 通过加密存储、校验和机制及端口保护等方式，已经加强了密码的保护，使得这种破解方法在现代系统中不再适用。

(3)（选做，部分指令知识需要预习）在长度为8的字节数组（无符号数）中，查找大于42H的无符号数的个数，存放在字节单元up中；等于42H的无符号数的个数，存放在字节单元equa中；小于42H的无符号数的个数，存放在字节单元down中。

代码

```
SM lab32.asm
1  data segment
2      array db 31H, 21H, 42H, 52H, 87H, 23H, 98H, 01H
3      up db 0
4      equa db 0      "equa": Unknown word.
5      down db 0
6
7      up_str db 'up: $'
8      equa_str db 'equa: $'      "equa": Unknown word.
9      down_str db 'down: $'
10     crlf      db 0DH, 0AH, '$'
11 data ends
12
```

数据段定义了数据 `array`，统计数据的 `up`、`equa`、`down`，存储格式字符串的 `up_str`、`equa_str`、`down_str` 和换行字符串 `crlf`

```

13  code segment
14      assume cs:code, ds:data
15  start:
16      mov ax, data
17      mov ds, ax
18
19      mov cx, 8
20      mov si, 0
21
22  loop_compare:
23      mov al, array[si]
24      cmp al, 42H
25
26      ja count_up
27      je count_equa      "equa": Unknown word.
28      jb count_down
29
30  count_up:
31      inc up
32      jmp next
33
34  count_equa:      "equa": Unknown word.
35      inc equa      "equa": Unknown word.
36      jmp next
37
38  count_down:
39      inc down
40      jmp next

```

这段code部分定义了条件判断跳转


```
41
42     next:
43         inc si
44         loop loop_compare
45
```

后面是格式输出的部分：

```
46         mov ah, 09H
47         lea dx, up_str
48         int 21H
49
50         mov ah, 02H
51         mov dl, up
52         add dl, '0'
53         int 21H
54
55         lea dx, crlf
56         mov ah, 09H
57         int 21H
58
```

输出up部分

```

59      lea dx, equa_str      "e
60      mov ah, 09H
61      int 21H
62
63      mov ah, 02H
64      mov dl, equa      "equa"
65      add dl, '0'
66      int 21H
67
68      lea dx, crlf
69      mov ah, 09H
70      int 21H
71

```

输出equa部分

```

72      lea dx, down_str
73      int 21H
74
75      mov ah, 02H
76      mov dl, down
77      add dl, '0'
78      int 21H
79
80      lea dx, crlf
81      mov ah, 09H
82      int 21H
83

```

输出down部分

```
83
84     mov ah, 4CH
85     int 21H
86 code ends
87
88 end start
```

程序结束

代码的全文如下：

```
1  data segment
2      array db 31H, 21H, 42H, 52H, 87H, 23H, 98H, 01H
3      up db 0
4      equa db 0
5      down db 0
6
7      up_str db 'up: $'
8      equa_str db 'equa: $'
9      down_str db 'down: $'
10     crlf db 0DH, 0AH, '$'
11 data ends
12
13 code segment
14     assume cs:code, ds:data
15 start:
16     mov ax, data
17     mov ds, ax
18
19     mov cx, 8
20     mov si, 0
21
22 loop_compare:
23     mov al, array[si]
24     cmp al, 42H
25
26     ja count_up
27     je count_equa
28     jb count_down
29
30 count_up:
```

```
31     inc up
32     jmp next
33
34 count_equa:
35     inc equa
36     jmp next
37
38 count_down:
39     inc down
40     jmp next
41
42 next:
43     inc si
44     loop loop_compare
45
46     mov ah, 09H
47     lea dx, up_str
48     int 21H
49
50     mov ah, 02H
51     mov dl, up
52     add dl, '0'
53     int 21H
54
55     lea dx, crlf
56     mov ah, 09H
57     int 21H
58
59     lea dx, equa_str
60     mov ah, 09H
61     int 21H
62
63     mov ah, 02H
64     mov dl, equa
65     add dl, '0'
66     int 21H
67
68     lea dx, crlf
69     mov ah, 09H
70     int 21H
71
72     lea dx, down_str
73     int 21H
74
```

```

75      mov ah, 02H
76      mov dl, down
77      add dl, '0'
78      int 21H
79
80      lea dx, crlf
81      mov ah, 09H
82      int 21H
83
84      mov ah, 4CH
85      int 21H
86  code ends
87
88  end start

```

运行结果

```

D:\>ml lab32.asm
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: lab32.asm

Microsoft (R) Segmented Executable Linker Version 5.60.339 Dec 5 1994
Copyright (C) Microsoft Corp 1984-1993. All rights reserved.

LINK : warning L4017: /r : unrecognized option name; option ignored
Object Modules [.obj]: /r lab32.obj
Run File [lab32.exe]: "lab32.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:
LINK : warning L4021: no stack segment

D:\>lab32.exe
up: 3
equa: 1
down: 4

D:\>

```

连接编译运行查看结果，得到up为3，equa为1，down为4，结果正确！

5.实验分析与总结

本次实验主要通过编写汇编语言程序并运用调试工具（如 `debug` 和 DOSBox）来加深对计算机底层操作的理解，涵盖了多个方面的内容，包括数据存储、寻址方式、汇编指令的使用、BIOS密码的破解以及字节数组处理等。

5.1 数据段存储与运算

实验的第一个部分是将一系列十六进制数存入数据段，并通过汇编指令进行加法运算，最后将结果存入 `DX` 寄存器。通过调试和检查寄存器的值，我们可以清楚地看到汇编程序正确地执行了加法运算，并正确地将结果存储在 `DX` 寄存器中。此次练习帮助加深了对数据段、寄存器和内存操作的理解，尤其是如何在汇编中进行基础的算术运算。

5.2 BIOS 密码破解

在实验的第二部分，我们使用了 `debug` 命令来模拟破解 BIOS 密码的过程。通过对硬件底层原理（如 CMOS 芯片的工作原理）和 I/O 端口的操作，我们能够了解如何通过修改 CMOS 存储的内容来绕过 BIOS 密码验证。通过实验，我们学到了密码存储的基本原理以及通过直接操作端口读取和修改密码的方式。此外，也了解了现代 BIOS 安全机制的发展，强调了简单的操作已无法破解现代 BIOS 密码。

5.3 数组元素比较与统计

实验的第三部分涉及到在一个字节数组中查找大于、等于、小于 `42H` 的无符号数个数，并分别存储在 `up`、`equa` 和 `down` 字节单元中。通过使用条件跳转和循环指令，我们能够遍历数组并对每个元素进行比较统计。这不仅巩固了汇编中常见指令（如 `cmp`、`ja`、`je`、`jb`）的应用，也加强了对汇编中数据存储和循环控制的掌握。通过输出结果，我们可以验证程序的正确性，确保各个部分的统计数据正确无误。

5.4 汇编语言的实践与理解

通过此次实验，我们不仅学习了如何在汇编中处理数据，还加深了对汇编语言基本语法和指令的理解。尤其是指令的使用、条件跳转、数据段的操作等，都是汇编编程中非常基础且重要的概念。此外，本次实验还帮助我们更好地理解虚拟机、调试工具的使用，以及如何进行低层次的硬件操作。

5.5 总结

通过本次实验，我对汇编语言和计算机硬件的底层操作有了更加深入的理解，掌握了汇编语言的一些常用指令，学会了如何在实际的调试环境中进行程序运行和调试。实验的过程中，我不仅巩固了基本的汇编知识，还学会了如何通过调试工具来检查程序的执行状态和结果。对于 BIOS 密码的破解理解也使我更深入地认识到计算机安全的基础知识和一些可能存在的安全漏洞。总体而言，本次实验不仅提升了我的汇编编程能力，也增强了我对计算机底层原理的理解。

